Sistema De Adquisición De Señal De Temperatura Con Sensor LM35 Y Filtros Digitales

Raquel Arias García, Diana Vanessa Guerra López, Claudia Elena Durango Cossio
Electrónica digital II

Índice

1. Identifiaciòn Del Proyecto **2**

2. Resumen **2**

3. Descripción del hardware **2**

4. Descripción del software **5**

5. Estructura del código **6**

6. Explicación de funciones principales **6**

7. Comunicación (si aplica) **7**

8. Interfaz de usuario (si aplica) **7**

9. Procedimiento de prueba **7**

10. Manejo de errores y seguridad **8**

1. Identifiaciòn Del Proyecto

- **Nombre del proyecto:** Sistema De Adquisición De Señal De Temperatura Con Sensor LM35 Y Filtros Digitales
- **Integrantes del equipo: Raquel Arias García, Diana Vanessa Guerra López, Claudia Elena Durango Cossio**

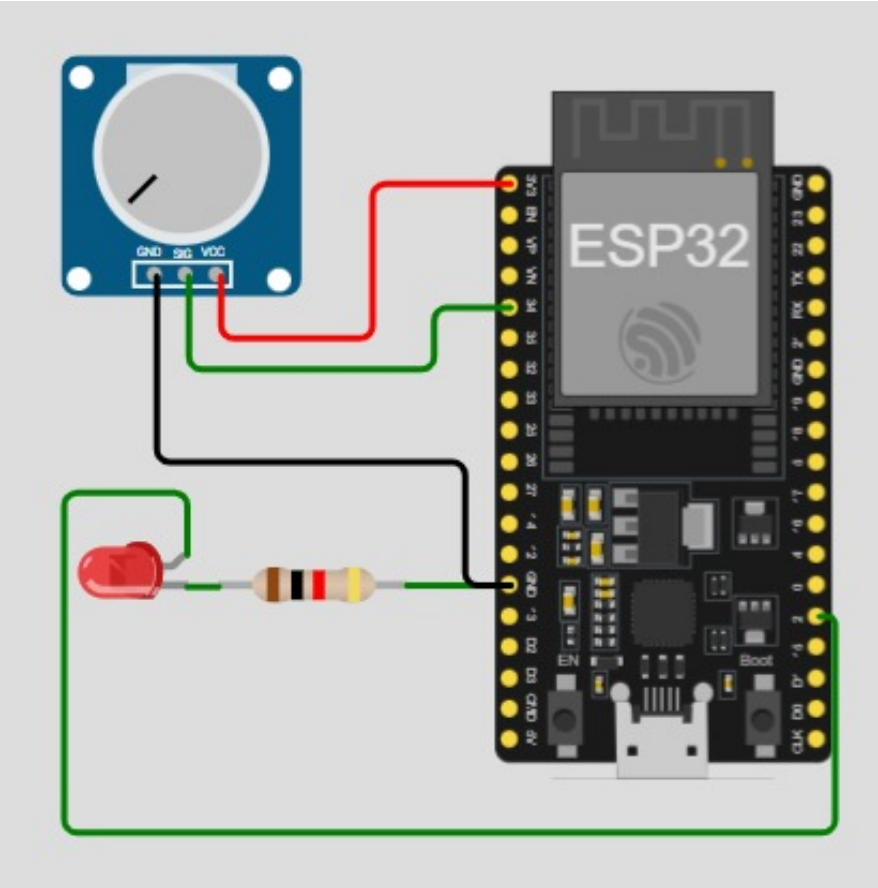
2. Resumen

En esta práctica se desarrolló un sistema de adquisición de temperatura utilizando un sensor LM35 y el microcontrolador ESP32. La señal analógica fue leída mediante el convertidor ADC y muestreada de forma periódica a una frecuencia definida por el usuario desde la terminal. Durante la ejecución, los datos fueron procesados en tiempo real aplicando diferentes filtros digitales, como promedio móvil, mediana y exponencial, los cuales podían configurarse de manera individual o en cascada al inicio del programa. La información obtenida, tanto sin filtrar como filtrada, se almacenó en un archivo de texto en la memoria interna del dispositivo y se mostró en la consola para su comparación. Este desarrollo permitió reforzar conceptos relacionados con adquisición de señales, temporización mediante interrupciones, uso de buffers y aplicación de técnicas básicas de filtrado digital.

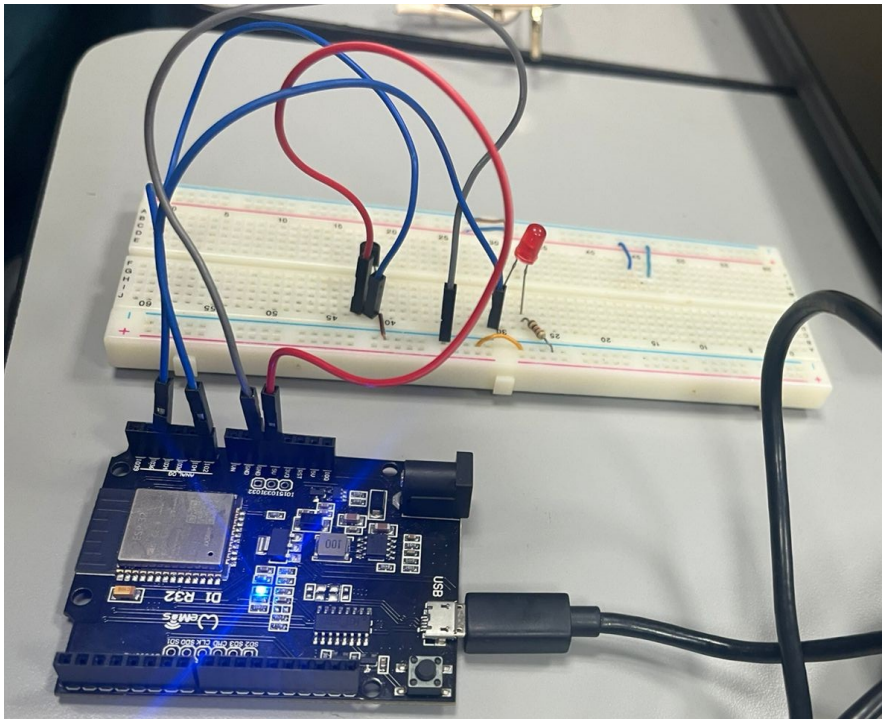
3. Descripción del hardware

- **El sistema se implementó utilizando el microcontrolador ESP32, el cual se encarga de la adquisición, procesamiento y almacenamiento de la señal de temperatura.**
- **Entradas:**
- **Señal analógica proveniente de un sensor de temperatura LM35, conectada al pin 34 configurado como entrada ADC**
- **Salidas:**
- **L**ED en el pin 2, el cual parpadea en cada muestreo como indicación visual de la frecuencia de adquisición.
- **A**rchivo de texto generado en la memoria interna del ESP32, donde se almacenan los datos adquiridos y procesados.
- **F**uente de alimentación:
- **A**limentación de 5V a través del puerto USB, garantizando un funcionamiento estable del sistema.

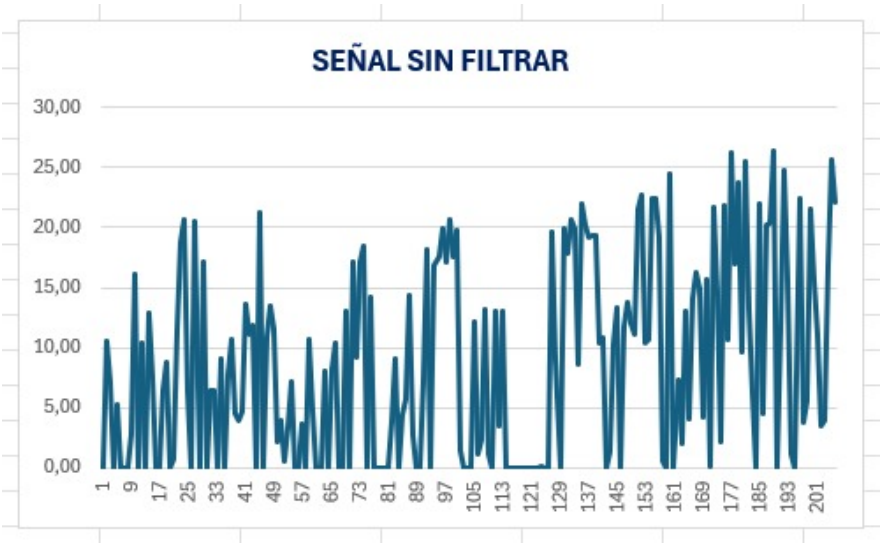
Diagrama de conexión:

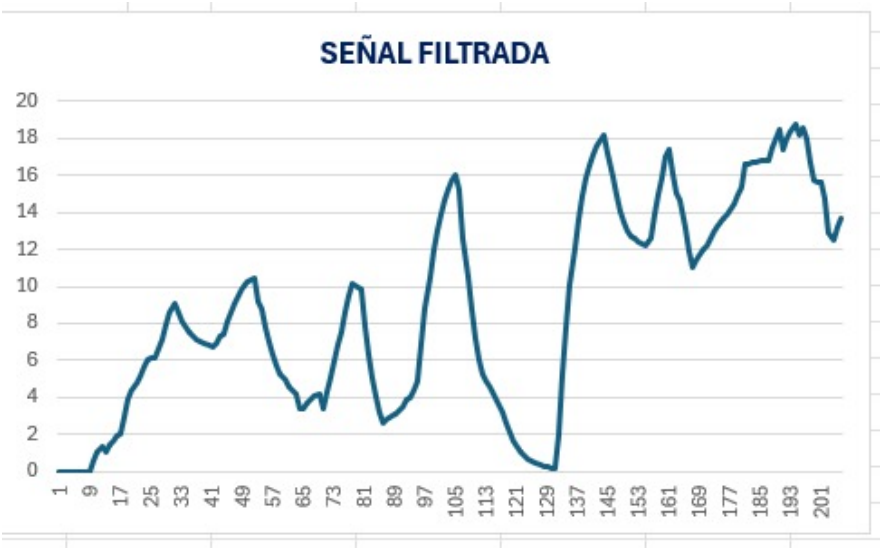


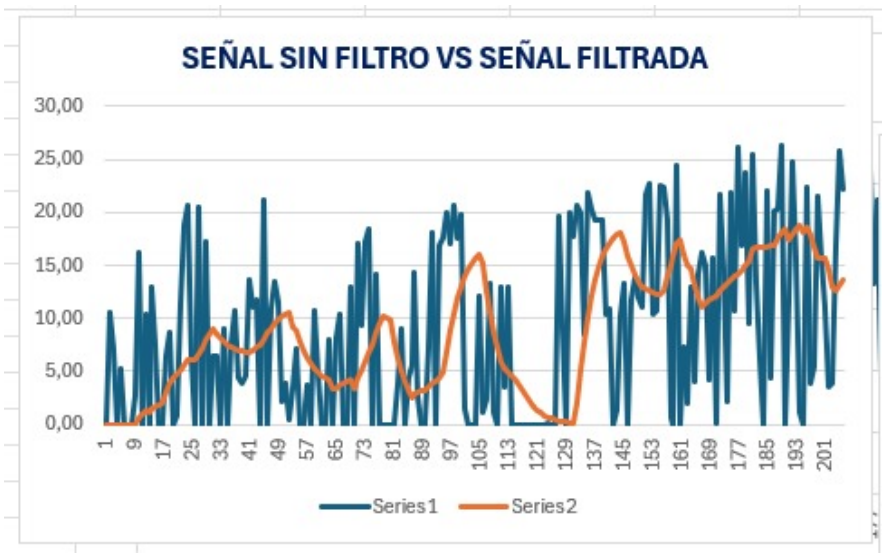
Montaje Fìsico :



Gràficas :


SEÑAL SIN FILTRAR
30,00
25,00
20,00
15,00
10,00
5,00
0,00
1 9 17 25 33 41 49 57 65 73 81 89 97 105 113 121 129 137 145 153 161 169 177 185 193 201


SEÑAL FILTRADA
20
18
16
14
12
10
8
6
4
2
0
1 9 17 25 33 41 49 57 65 73 81 89 97 105 113 121 129 137 145 153 161 169 177 185 193 201



4. Descripción del software

- **Lenguaje usado:** MicroPython
- **Librerías:** `machine`, `time` `uos`,
- **IDE:** Thonny

Flujo general del programa:

1. Configuración del pin ADC para la lectura de la señal analógica proveniente del sensor LM35.
2. Configuración del pin de salida (LED) como indicador visual del proceso de muestreo.
3. Definición de la frecuencia de muestreo ingresada por el usuario y establecimiento del tamaño del buffer circular para el almacenamiento de datos.
4. Implementación de los filtros digitales: promedio móvil, mediana y exponencial.
5. Selección de los filtros a aplicar mediante un menú en la consola al inicio de la ejecución.
6. Creación de un archivo de texto en la memoria interna del ESP32 para el almacenamiento de los datos.
7. Uso de un temporizador para realizar el muestreo periódico de la señal y actualizar el buffer con cada nueva lectura.
8. Aplicación de los filtros seleccionados en tiempo real y visualización de los resultados, tanto en consola como en el archivo generado.

5. Estructura del código

- Archivo principal: `main.py`
- Variables globales: Parámetros de muestreo (frecuencia y periodo), buffer circular de lecturas, índice de posición, variables para el filtro exponencial, bandera de nueva muestra y última lectura del ADC.
- Funciones auxiliares:
- promedio(): calcula el valor promedio de los datos almacenados en el buffer.
- mediana(): obtiene el valor central del buffer ordenado.
- exponencial(valor): aplica un suavizado exponencial a la señal a partir del valor actual.
- $\text{adc}_{at}emp(valor) : convierte$
- el valor leído del ADC a temperatura en grados Celsius
- Interrupciones:
- muestrear(): función de callback asociada al temporizador, encargada de adquirir una nueva lectura del ADC según la frecuencia definida por el usuario y alternar el estado del LED como indicador de muestreo.
- Lógica principal:
- 1. Selección de los filtros a aplicar por parte del usuario desde la consola.
- 2. Inicialización del archivo de almacenamiento y escritura del encabezado.
- 3. Ejecución de un ciclo infinito que, al detectar una nueva muestra, actualiza el buffer, aplica los filtros seleccionados en el orden definido y almacena los resultados junto con la señal sin filtrar.

6. Explicación de funciones principales

- promedio(): reduce el ruido presente en la señal al calcular el promedio de los valores almacenados en el buffer, generando una señal más suave y estable.
- mediana(): permite eliminar valores atípicos o picos espurios, seleccionando el valor central del conjunto de datos ordenados dentro del buffer.
- exponencial(valor): aplica un suavizado progresivo a la señal, dando mayor peso a los valores más recientes mediante un factor de ajuste, lo que mejora la respuesta ante cambios.
- `muestrear`(timer): función asociada al temporizador que realiza la lectura del ADC según la frecuencia de muestreo definida por el usuario, actualiza la última muestra disponible y cambia el estado del LED como indicación del proceso de adquisición.
- selección de filtros: se realiza al inicio del programa mediante la consola, donde el usuario define cuántos filtros desea aplicar y en qué orden, permitiendo su uso individual o en cascada durante la ejecución.

7. Comunicación (si aplica)

La comunicación en el sistema se realiza a través de dos mecanismos principales:

Consola serial: permite visualizar en tiempo real los valores de temperatura sin filtrar y filtrados. Estos datos se envían en un formato adecuado para su representación en herramientas como el Serial Plotter, facilitando la comparación entre la señal original y la procesada. Archivo de texto: se genera un archivo local en la memoria interna del ESP32, donde se almacenan los datos adquiridos de cada muestra, incluyendo tanto la señal cruda como la filtrada. Esto permite realizar un análisis posterior de la información registrada.

8. Interfaz de usuario (si aplica)

El sistema cuenta con una interfaz basada en consola que permite la interacción directa con el usuario. Al iniciar el programa, se presenta un menú donde se puede seleccionar la cantidad de filtros a utilizar y el orden en que se aplicarán, facilitando su uso individual o en cascada.

Como indicador visual del proceso de adquisición, se emplea un LED conectado al pin 2, el cual cambia de estado en cada muestreo, permitiendo verificar el funcionamiento del sistema.

Adicionalmente, se genera un archivo de salida en la memoria interna del dispositivo, donde los datos se organizan en dos columnas correspondientes a la señal sin filtrar y la señal filtrada, lo que permite su posterior análisis y comparación.

9. Procedimiento de prueba

1. Conectar el ESP32 al computador mediante cable USB y abrir el entorno de desarrollo.
2. Cargar y ejecutar el archivo main.py.
3. Ingresar la frecuencia de muestreo solicitada desde la consola
4. Seleccionar los filtros a aplicar y definir su orden de ejecución.
5. Verificar que el LED parpadea de acuerdo con la frecuencia de muestreo configurada.
6. Observar en la consola serial los valores de temperatura sin filtrar y filtrados.
7. Abrir el archivo temp$_l m35.txt$
7. Confirmar que los datos se están almacenando correctamente.
8. Detener la ejecución del programa utilizando la interrupción por teclado (Ctrl + C).

10. Manejo de errores y seguridad

- El sistema incluye validaciones básicas para la entrada de datos desde la consola, especialmente en la selección de la frecuencia de muestreo y de los filtros, evitando fallos por valores inválidos.
- Se implementa un buffer circular para el almacenamiento de las muestras, lo que permite un uso eficiente de la memoria y evita desbordamientos durante la ejecución continua del programa.
- El uso de un temporizador para el muestreo garantiza una adquisición periódica y estable de la señal, evitando bloqueos en el ciclo principal y mejorando la confiabilidad del sistema.